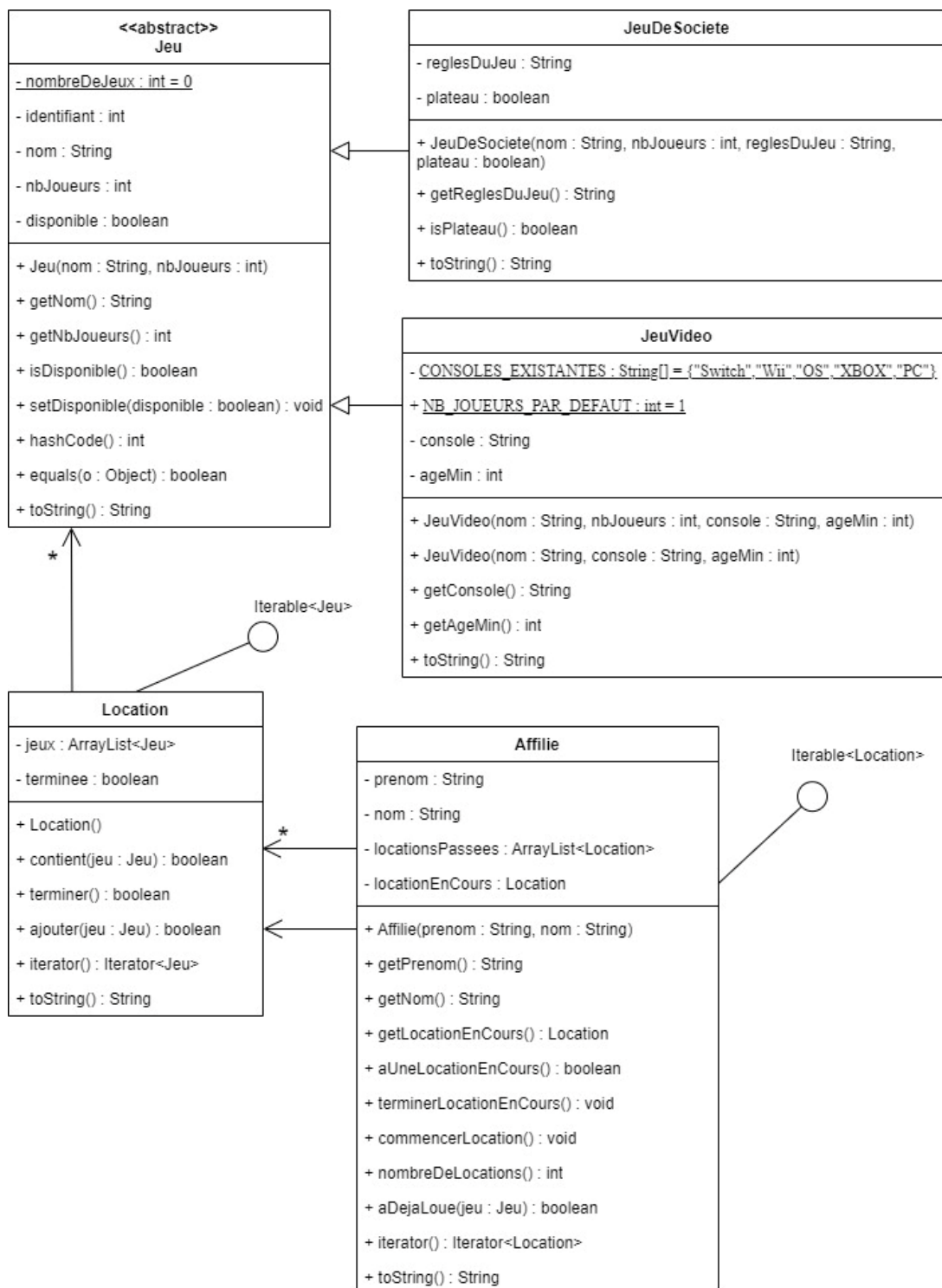


EXERCICES RÉCAPITULATIFS – PARTIE 1

1. Gestion d'une ludothèque

Vous trouverez, sur moodle, le début de la classe Jeu ainsi qu'un fichier avec les méthodes toString à compléter. Sur base du diagramme de classes (point a), de la représentation mémoire (point b) et des consignes associées, l'objectif est de créer un programme permettant la gestion des affiliés, des jeux et des locations d'une ludothèque.

a) Implémentation des classes



Contexte : Une ludothèque permet à ses affiliés de louer des jeux (des jeux de société ou des jeux vidéo). Celle-ci aimerait donc que vous lui fournissiez un programme lui permettant de gérer les locations de ses affiliés.

Ne lancez des exceptions que lorsque c'est explicitement demandé.

Veuillez n'implémenter les getters et setters que pour les cas spécifiés au sein du diagramme de classes. Veuillez à respecter les bonnes pratiques de programmation vues lors des séances d'exercices.

Pour les méthodes toString des différentes classes, vous trouverez des indications dans la partie b et des squelettes de codes dans le fichier toStrings.txt

Récupérez le début de la classe **Jeu** fournie dans laquelle se trouve déjà certains attributs. Modifiez, si nécessaire, son en-tête et ajoutez-y le ou les attributs manquants ainsi que les méthodes demandées :

- Le constructeur **Jeu**, ne prend que le nom du jeu et le nombre de joueurs en paramètre. L'identifiant sera automatiquement attribué par la classe (le premier jeu créé doit avoir l'identifiant 1, le deuxième l'identifiant 2, ...) et sera disponible.
- Un jeu est identifié par son identifiant.

Pour la classe **JeuVideo** :

- Lorsque le paramètre console n'est pas présent dans la table CONSOLES_EXISTANTES, les constructeurs lance une IllegalArgumentException avec le message « XXX n'est pas une console valide » où XXX doit être remplacé par la console passée en paramètre.
- Le constructeur à 3 paramètres initialise l'attribut nbJoueurs avec la valeur contenue dans la variable NB_JOUEURS_PAR_DEFAULT.

Pour la classe **JeuDeSociete** :

- L'attribut plateau indique si le jeu de société est un jeu de plateau ou pas (true si c'est un jeu de plateau et false sinon).

Pour la classe **Location** :

- Le constructeur crée une location vide (sans jeu) et non terminée.
- La méthode terminer renvoie faux si la location est déjà terminée. Sinon, elle rend disponible tous les jeux de la location et renvoie vrai.
- La méthode contient renvoie vrai si la location contient le jeu passé en paramètre et faux sinon.
- La méthode ajouter échoue (renvoie faux) si le jeu n'est pas disponible ou si la location est terminée. Sinon elle ajoute le jeu dans la liste des jeux loués, indique que le jeu n'est plus disponible et renvoie vrai.

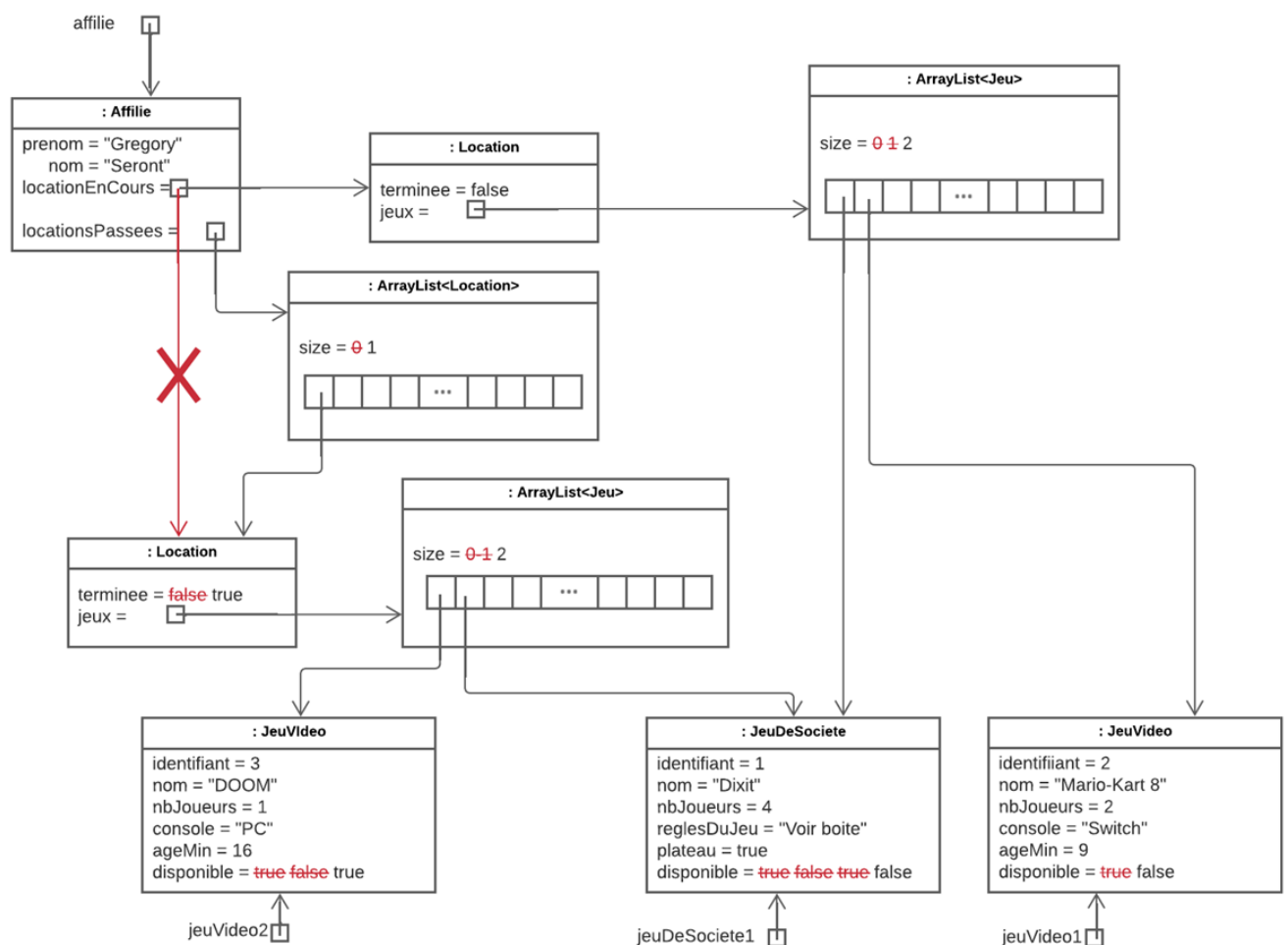
Pour la classe **Affilie** :

- Le constructeur crée un affilié en initialisant ses attributs prenom, nom avec les valeurs passées en paramètre. À sa création, un affilié n'a pas encore de locations passées, ni de location en cours.
- La méthode aUneLocationEnCours renvoie vrai si l'affilié a une location en cours et faux sinon.
- La méthode terminerLocationEnCours lance une IllegalStateException avec le message « Pas de location en cours » si l'affilié n'a pas de location en cours. Sinon elle termine la

location en cours, l'ajoute dans la liste des locations passées de l'affilié et supprime la location en cours.

- La méthode commencerLocation lance une `IllegalStateException` avec le message « Il y a déjà une location en cours » si l'affilié a déjà une location en cours. Si aucune exception n'est lancée alors cette méthode initialise la location en cours en créant une nouvelle location ne contenant aucun jeu.
- La méthode nombreDeLocations renvoie le nombre de locations faites par l'affilié en comptant celle en cours s'il y en a une.
- La méthode aDejaLoue renvoie `true` si le jeu a déjà été pris en location par le membre par le passé et `false` sinon.
- La méthode iterator renvoie un itérateur permettant de parcourir uniquement les locations passées.

b) Implémentation du programme principal



Créez votre classe **TestLudotheque**, sur base de la représentation des objets en mémoire et des instructions suivantes. Veillez aussi à ce que les méthodes **toString** de vos différentes classes fassent en sorte que les affichages correspondent à ceux indiqués dans les scénarios ci-dessous. Pour cela, récupérez les **toString** fournis dans le fichier `toStrings.txt`, copiez-les dans les bonnes classes et complétez-les en tenant compte des commentaires

Au sein de votre code, veuillez introduire chacun des scénarios de tests que vous implémentez par un commentaire reprenant le numéro de l'étape indiqué ci-dessous.

Étape 1 : création d'instances

Veillez créer les instances des différents jeux (en veillant à utiliser le constructeur le plus approprié) ainsi que l'affilié apparaissant dans la représentation mémoire de la figure 2. Affichez ensuite l'état du jeuDeSociete1 et de l'affilié dans la console. L'affichage attendu est le suivant :

```
Jeu de plateau 1 disponible : Dixit pour 4 joueur(s) dont les règles sont
Voir boîte
Gregory Seront
Pas de location en cours
Liste des locations passées :
-----
Pas encore de location passée
```

Étape 2 : ajout d'une location à l'affilier

Tentez de terminer la location en cours de l'affilié et affichez le message contenu dans l'exception dans la console. Ensuite veuillez faire commencer la première location de l'affilié et ajoutez-lui les jeux loués selon la représentation mémoire. Affichez ensuite l'état de l'affilié. L'affichage attendu est le suivant :

```
Pas de location en cours
Gregory Seront
Location en cours :
-----
Location en cours :
1) Jeu vidéo 3 loué : DOOM pour 1 joueur(s) sur PC pour un age minimum de 16 ans
2) Jeu de plateau 1 loué : Dixit pour 4 joueur(s) dont les règles sont
Voir boîte
Liste des locations passées :
-----
Pas encore de location passée
```

Étape 3 : clôture de la location en cours et nouvelle location

Terminez la location en cours de l'affilié. Ensuite commencez une nouvelle location pour l'affilié et ajoutez-lui les jeux loués selon la représentation mémoire. Enfin affichez le nombre de locations et l'état de l'affilié. L'affichage attendu est le suivant :

```
2
Gregory Seront
Location en cours :
-----
Location en cours :
1) Jeu de plateau 1 loué : Dixit pour 4 joueur(s) dont les règles sont
Voir boîte
2) Jeu vidéo 2 loué : Mario-Kart 8 pour 2 joueur(s) sur Switch pour un age
minimum de 9 ans
Liste des locations passées :
-----
Pret terminé :
1) Jeu vidéo 3 disponible : DOOM pour 1 joueur(s) sur PC pour un age minimum de
16 ans
2) Jeu de plateau 1 loué : Dixit pour 4 joueur(s) dont les règles sont
Voir boîte
```

2. Gestion d'un championnat de football

Une fédération de football veut clarifier son site de gestion des clubs de football. Votre expertise est requise pour établir le diagramme des classes de son domaine.

L'application est centrée sur les clubs de football (Club) et les résultats des championnats entre ses clubs.

Un club possède un matricule (un entier attribué automatiquement lors de l'instanciation : le premier club créé aura le numéro 1, le deuxième 2, ...), un nom, un entraîneur, une liste de joueurs, un terrain et un terrain d'entraînement. Pour instancier un club, son nom et son terrain sont indispensables. Un terrain d'entraînement peut être aussi spécifié lors de l'instanciation ou ajouté par la suite. Un club peut engager un entraîneur, licencier un entraîneur, engager ou licencier un joueur. Enfin on peut parcourir les joueurs du club (avec un foreach) et connaître leur nombre. Attention un club ne peut pas engager un joueur/entraîneur si celui-ci est déjà dans un club.

Un membre de la fédération possède un nom, un prénom et une nationalité. On doit pouvoir vérifier s'il est déjà dans un club, préciser qu'il a été engagé par un club ou encore dire qu'il a été licencié. Il n'est pas possible d'instancier un membre. Un membre est soit un joueur soit un entraîneur.

Un joueur possède une position sur le terrain qui peut être changée. Il faut vérifier que la position corresponde à une position permise à savoir : « G » pour gardien, « D » pour défenseur, « M » pour milieu et « A » pour attaquant. Si ce n'est pas le cas, il faut lancer une `IllegalArgumentException`. Un entraîneur possède une tactique (sous forme de chaîne de caractères). Que ce soit un joueur ou un entraîneur, il ne fait pas partie d'un club lors de sa création. Les autres informations du joueur (ou d'un entraîneur) seront fournies lors de sa création.

Un terrain possède une longueur, une largeur. Il faut aussi savoir si la pelouse est naturelle ou non et connaître la date du dernier renouvellement de la pelouse. Lors de la création du terrain, ses mesures et le fait de savoir si la pelouse est naturelle ou non sont nécessaires. Par contre, la date de renouvellement correspond au moment de cette création. On peut changer la pelouse du terrain. Lors du changement, il faut préciser si la nouvelle pelouse est naturelle ou non.

Un championnat comporte un nom et la liste des équipes. On peut commencer le championnat, ajouter une équipe au championnat (tant que celui-ci n'a pas commencé), vérifier si le championnat a commencé, terminer le championnat, donner l'équipe qui est en tête du championnat (si celui-ci a commencé), donner le nombre d'équipes et donner le classement sous forme de texte. Pour instancier un championnat, seul son nom est nécessaire.

Une équipe comporte un club et un nombre de points. Seul le club est nécessaire lors de l'instanciation, les points étant mis à 0 automatiquement. On doit pouvoir ajouter un nombre de points à l'équipe.

Donnez le diagramme de classes UML de cette application. Vous ne devez pas mettre les getters, ni les toString.